

Plane Identification for Underwater Tactile Sensors

Eu Joe Wong - 21104713

Department of Mechanical Engineering, University College London

ABSTRACT

In challenging deep-sea environments, tactile sensors can complement underwater cameras, particularly in vision-limited scenarios where light is scarce and pressures are extreme. This study focuses on the development of a program that simulates a tactile sensor's ability to identify planes in 3D space and investigates how parameters such as number of contact points on the sensor (α), number of readings (β), and noise distributions affect shape accuracy. Using Python, a simulation environment and an identification program was created. Tactile sensor data was simulated by defining a known plane in the environment and by introducing noise on the plane's surface to simulate surface imperfections such as marine biofouling. An independent identification program then interprets the data by employing principal component analysis to output a best-fit plane and its location in space. Performance metrics such as angle difference between the real and fitted plane (θ), distance between the planes (δ), and coefficient of determination (R^2) were analysed. Out of all test cases, the highest angle difference recorded after 1 reading was $\theta = 0.18 \text{ rad}$ with Gaussian noise at $\alpha = 3$. The largest distance between planes recorded after 1 reading was $\delta = 0.89 \text{ cm}$ at $\alpha = 4$ again subject to Gaussian noise. Results showed high identification accuracy, with a greater number of sensor contact points leading to faster convergence of the best-fit plane. The identification program also displayed lower θ values with a Uniform noise distribution and lower δ values with a Gaussian distribution.

Keywords: plane identification, best-fit, tactile sensor, marine biofouling, principal component analysis

NOMENCLATURE

α	number of linear sensors	k	maximum noise height
β	number of readings	k_s	average noise height
δ	distance from real to fitted plane	$\hat{n}$	unit vector
θ	angle between real and fitted plane	$\vec{p}$	point in space
$\mathbf{A}$	matrix of point coordinates	R^2	coefficient of determination
$\mathbf{B}$	mean-centred matrix	$\vec{v}$	3D vector

1. INTRODUCTION

Touch and sight are two sensory modalities most often used to identify objects. If sight becomes somehow limited, the reliance on touch is greatly increased. In many scenarios, humans tend to rely on touch rather than sight, either due to restricted visuals or for practical purposes, such as locating keys in a bag. Similarly, in deep sea conditions where light is virtually non-existent, it is difficult to obtain information from underwater cameras alone. Especially when moving along the seabed, nutrient-rich sediments will rise and form dust clouds that further obstruct the view [1]. Therefore, the implementation of tactile sensors can aid in object identification under such extreme conditions.

1.1 AIM

The aim is to develop a program that can simulate how a real-world tactile sensor would identify

planes in 3D. This simulation will then be used as a tool to investigate how the accuracy of the detected shape is affected when changing various parameters. Noise will be introduced to simulate surface imperfections and statistical techniques will be discussed to effectively identify the plane at hand while accounting for uncertainties. Final identification results will then be collected and analysed. All work will be simulation via mathematical modelling and the programming language used is Python. The hope is that this project will improve the understanding of tactile identification.

2. LITERATURE REVIEW

Tactile sensing was largely neglected in the early phases of robotics as only a handful of devices were developed by the end of the 1970s [2]. Integration of such devices into robots were also very limited.

In the 1980s, significant progress was made in sensing materials and fabrication alongside reduced manufacturing costs [2]. This resulted in substantial advancements in tactile sensing technology. In recent years, tactile sensors for underwater use have been developed especially for murky and turbid environments [3]. Although the implementation of underwater tactile sensing is relatively new, extensive research has been conducted regarding object recognition. One method that researchers from Università di Parma have focused on is to represent the explored objects as volumetric models that are then compared with a known object database [4]. A model used by the researchers is the *Enveloping Polyhedral Model* (EPM), which is an upper approximation of the object. At the start of the exploration, the EPM is equivalent to all of space. As data is gathered from contact points on the object, contact planes are generated that “slice” space into smaller volumes. This eventually results in a volume approximation of the object. Additionally, a neural network was also implemented in the recognition and matching process of the unknown object to obtain improved results. Many researchers have also adopted machine learning for tactile identification, such as the Bag-of-Features approach using low-resolution images generated from tactile feedback [5]. Another approach is the use of self-organizing maps to identify a set of household objects and toys [6]. Although work done in object identification has been extensive and is often paired with machine learning models, this project will focus on the fundamental basics of shape identification using a device with cylindrical linear sensors attached on its surface as shown in **Figure 1**.

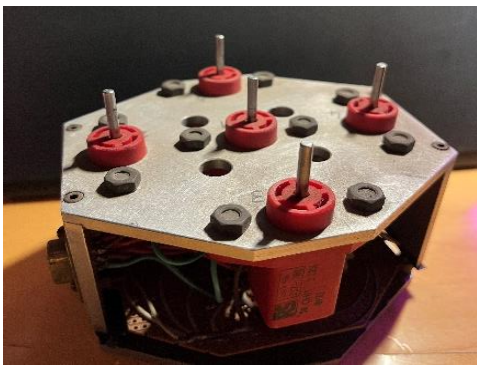


Figure 1: Sample tactile sensor consisting of 5 linear sensors used in PhD research [7].

Upon contact with external objects, the linear sensors compress into the device. The compression distance can then be measured and recorded for further data analysis.

3. METHODOLOGY AND SET-UP

No physical tactile sensor is used in this project. Instead, performing all simulations in Python will allow for easy changes in variables and rapid script iterations. However, to test and obtain results from a program that identifies shapes, data is required. Therefore, a completely separate program that can mimic real sensor data to feed into a shape identification program is desired. In the context of a tactile sensor, the simulated data will be the coordinates of the contact points in space that a real sensor would measure. Both the data simulation program and the identification program will work together to mirror a real-world tactile sensor. The general methodology listed below:

1. A known plane is defined in 3D space.
2. Using the known shape, coordinates are generated that simulate the contact points a real sensor would measure on the shape. Artificial noise is also introduced to simulate real-life measurements.
3. An identification program is created to interpret the contact coordinates provided. The identification program then outputs the detected shape, and its location in 3D space.
4. How well the detected shape matches the initially defined shape is analysed.

The effect of the number of linear sensors, number of readings, and different types of noise will also be analysed. After each step in the methodology, the validity of the program will be compared against known results before moving on to the next stage. When finished, validation and verification of the complete program will take place, followed by data collection. After analysis of the results, the goal is to determine the most robust and appropriate sensor setup that provides sufficiently accurate identification results in the presence of external noise.

3.1 Program Structure

In the real world, there are three main points to consider when utilizing tactile sensing. The first is the environment. This includes the region in which the sensor operates, and any objects of interest that are present. The second is data collection, the role of the tactile sensor. Its purpose is to obtain data through physical contact with the object of interest. Finally, there is data analysis. The data collected from the sensor is interpreted to output the measured object.

In this study, there is one program that both simulates the environment and performs data collection. It is named *planeddefine*. *planeddefine* creates an environment with a plane and provides sensor data using the given plane in space. *planeddefine* simulates how a real tactile sensor would obtain measurements. For data analysis, a program is created called *IDS*, which is short for ‘shape identification’. The purpose of *IDS* is to interpret the data given from *planeddefine* and correctly identify the shape and its location in space. To summarize, *planeddefine* creates the environment and collects data, while *IDS* analyses the data and attempts to output the correct shape.

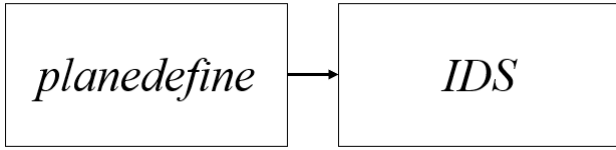


Figure 2: Block diagram of program structure.

Note that although *IDS* uses data from *planeddefine*, both scripts are completely independent of each other.

3.2 Sensor Arrangement

The tactile sensor to be modelled is shown below.

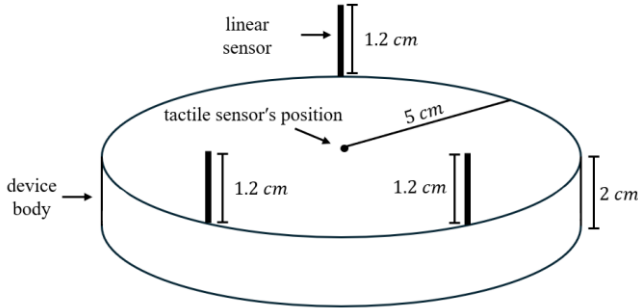


Figure 3: Schematic of tactile sensor dimensions. The point on the top surface represents its “position” in space. The thick lines represent the linear sensors extending from the top surface.

The sensor is in the shape of a circular plate. On one side, linear sensors are installed with various configurations. The simplest sensor configuration will be three equidistant sensors located along the plate’s circumference, as shown in **Figure 3**. The effect of different configurations will be explored in later sections. All sensors protrude outwards and are normal to the plate with an uncompressed length of 1.2 cm. The dimensions of the whole tactile sensor will remain constant throughout the project.

3.3 Sensor Assumptions

To simplify the scope of work, several assumptions regarding the tactile sensor are made:

1. The sensor has dimensions shown in **Figure 3**.
2. When a linear sensor comes into contact with an external object, its cylindrical rod can only compress vertically into the plate.
3. The maximum compression of each sensor is 1.2 cm.
4. The dimensions of each linear sensor are assumed to be negligible compared to the objects of interest. *planeddefine* will therefore treat the linear sensors as lines extending from the circular plate.
5. Since the dimensions of the linear sensors are negligible, the contact surfaces of the tip of the rods will be treated as contact points.
6. The tactile sensor is able to freely move and orientate itself to any physically viable position and orientation underwater.
7. Its position (shown in **Figure 3**), pitch, roll, and yaw are known at all times.
8. The compression displacements of the linear sensors can be measured with no error.
9. A successful reading from the tactile sensor occurs when each sensor is in contact with the object.
10. If a reading is successful, *planeddefine* will output the coordinates of the contact points of each linear sensor.
11. If a reading is not successful, the tactile sensor will reposition itself until it is successful.

3.4 Variables Changing

After *IDS* is created, its results will be analysed by changing the variables listed below.

- α : Number of linear sensors evenly spaced around sensor circumference.
- β : Number of readings.
- Type of noise: Gaussian/Uniform/Poisson

These variables affect how contact point coordinates are generated in the *planeddefine* script, thereby influencing results obtained from *IDS*.

4. SENSOR DATA SIMULATION

This section will outline how *planeddefine* is created. The procedure described will be for a three-linear-sensor configuration ($\alpha = 3$). However, the general procedure is also valid for increased α .

Three random points in space are first generated within the environment to define a plane with the form:

$$n_1x + n_2y + n_3z = d \quad (1)$$

Where n_1 , n_2 , and n_3 are the components of the plane's unit normal and d is the shortest distance from the plane to the origin. This will be the plane that *IDS* will attempt to identify. Next, *planedefine* will use this plane to simulate sensor contact points.

Let $\bar{v}_{sensor}$ be a unit vector that points in the direction of the sensor's orientation, $\hat{n}_{real}$ be the unit normal of the known plane in space, and the angle between the two vectors be θ .

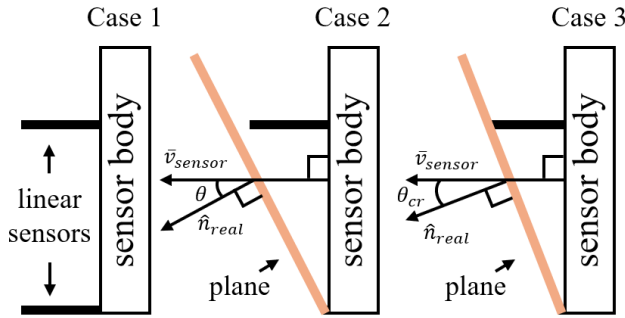


Figure 4: Side view of three possible cases for the three-linear-sensor configuration. In Case 1, no sensors are compressed. In Case 2, not all sensors are in contact. In Case 3, all sensors are in contact.

From **Figure 4** above, only Case 3 is a successful reading because it fulfils Assumption 8. Thus, there must be a critical angle θ_{cr} such that any angle above this will not result in a successful reading. For example, in Case 2, $\theta > \theta_{cr}$ and not all sensors are in contact with the plane. Therefore, there is a 'conical region' of allowable $\bar{v}_{sensor}$ vectors with respect to $\hat{n}_{real}$ that will result in a successful reading.

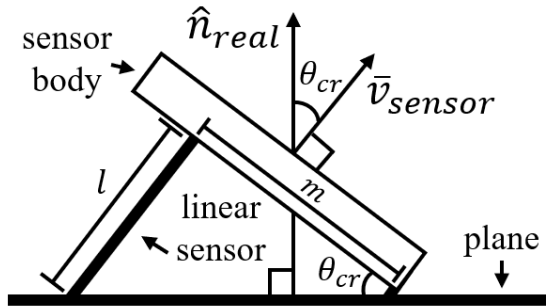


Figure 5: Side profile of all three sensors in contact with the plane. This is a critical case in which two sensors are fully compressed, while the third just touches the plane.

For a three-linear-sensor configuration, the critical angle is:

$$\theta_{cr} = \tan^{-1}\left(\frac{l}{m}\right) = \tan^{-1}\left(\frac{l}{k(1+\cos(\frac{\pi}{3}))}\right) \quad (2)$$

$$\theta_{cr, \alpha=3} = 0.159 \text{ rad}$$

The critical angles for different number of sensors are tabulated below.

Number of Sensors α	θ_{cr} [rad]
3	0.159
4	0.119
5	0.132

Table 1: Different critical angles for different number of sensors evenly spaced around the edge of the body.

Now that the critical angles are known, *planedefine* can generate random sensor vectors that have an angle less than θ_{cr} with respect to $\hat{n}_{real}$. This ensures that the sensor orientation will always provide a successful reading. To achieve this, a random vector $\bar{v}_{rand1}$ is generated. Next, a random rotation axis $\bar{v}_{rot}$ is defined by:

$$\bar{v}_{rot} = \frac{\hat{n}_{real} \times \bar{v}_{rand1}}{|\hat{n}_{real} \times \bar{v}_{rand1}|} \quad (3)$$

$\bar{v}_{rot}$ is perpendicular to $\hat{n}_{real}$ and provides a random axis in which $\hat{n}_{real}$ can be rotated about. The *Rodrigues' Rotation Formula* is then used to rotate $\hat{n}_{real}$ by a random angle $-\theta_{cr} < \theta_{rand} < \theta_{cr}$ about the axis $\bar{v}_{rot}$:

$$\begin{aligned} \bar{v}_{sensor} = & \hat{n}_{real} \cos(\theta_{rand}) \\ & + (\bar{v}_{rot} \times \hat{n}_{real}) \sin(\theta_{rand}) \\ & + \bar{v}_{rot} (\bar{v}_{rot} \cdot \hat{n}_{real}) (1 - \cos(\theta_{rand})) \end{aligned} \quad (4)$$

Thus, $\bar{v}_{sensor}$ is simply $\hat{n}_{real}$ rotated about a random axis $\bar{v}_{rot}$ by a random angle θ_{rand} .

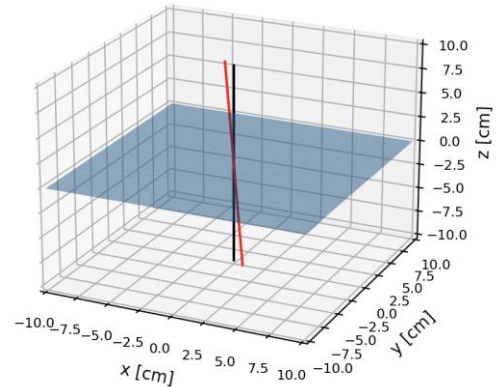


Figure 6: 3D plot of the real plane, its unit normal $\hat{n}_{real}$ (black line), and the randomly generated sensor vector $\bar{v}_{sensor}$ (red line).

Once $\bar{v}_{sensor}$ is created, a second random vector $\bar{v}_{rand2}$ is generated. Using $\bar{v}_{rand2}$, a vector $\bar{v}_p$ is

created that extends orthogonally from $\bar{v}_{sensor}$ to one of the linear sensors with a magnitude of 5 cm (this is the distance from the centre of the device to the linear sensor).

$$\bar{v}_p = 5 \frac{\bar{v}_{sensor} \times \bar{v}_{rand2}}{|\bar{v}_{sensor} \times \bar{v}_{rand2}|} \quad (5)$$

Next, a separate vector perpendicular to $\bar{v}_p$ and parallel to $\bar{v}_{sensor}$ is created to form a line that passes along the first linear sensor. Since the three sensors are a known distance apart, two additional parallel vectors can be created to represent the axis of the two other sensors. The intersection points of these three vectors with the plane can be computed. To simulate surface imperfections, noise is then introduced along the axis of each sensor to shift the points. The goal of the project is to effectively identify planes by analysing how well *IDS* accounts for this noise.

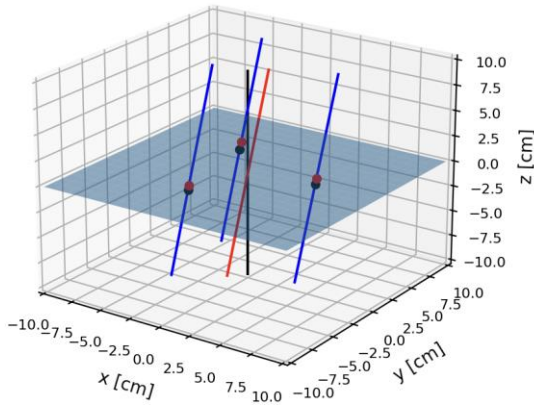


Figure 7: 3D plot of the real plane, $\hat{n}_{real}$ (black line), $\bar{v}_{sensor}$ (red line), and the three lines that represent the axis of the three linear sensors (blue lines). The points at which the blue vectors intersect the plane are highlighted in black. The red points are the result of the black points shifted due to noise.

From **Figure 7**, the coordinates of the three red points are what *planedefine* will output. This is simulated data that a real tactile sensor would provide if it came into contact with a plane.

4.1 Simulating Noise

This section will address how the red coordinates depicted in **Figure 7** are obtained to simulate noise. There are two main sources of noise when obtaining measurements with the tactile sensor. The first is measurement error from the sensors themselves. Although most modern contact displacement sensors, such as LVDTs, are able to achieve errors in the range of microns, their measurement ranges are extremely small ($< 1 \text{ mm}$) and are more suited

for metrological purposes [8]. A more suitable sensor for this study is a pneumatic displacement sensor. There are various commercial pneumatic sensors that feature a compression range of around 1.2 cm. For example, Festo, a German automation company, produces cylindrical linear sensors with errors up to $\pm 0.008 \text{ cm}$ for displacements less than 5 cm [9]. Studies have also shown that pneumatic actuators can be viable for underwater applications, especially if integrated with soft robotics [10].

The second main source of noise is surface imperfections on the object of interest, which can result from various factors such as marine biofouling, coating roughness, or surface cracks [11] [12]. In a recent experimental study of surface roughness effects of a submerged floating tunnel, researchers modelled biofouling with a maximum roughness height of $k = 1.5 \text{ cm}$ [13]. A repeating sequence of pyramids was used to model the roughness element and the average roughness height used was $k_s = 0.75 \text{ cm}$ [13]. Both k and k_s are defined in **Figure 8** below.

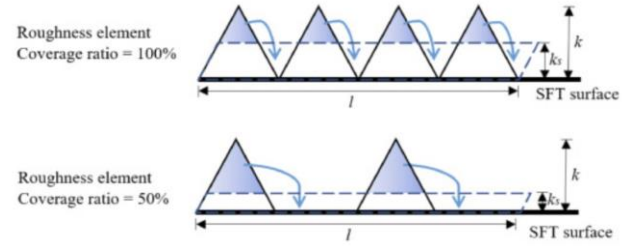


Figure 8: Example of marine biofouling model using a pyramidal pattern [13].

The accuracy of the noise model is of secondary importance, as the primary goal is to introduce different noise distributions to evaluate the robustness of the identification program. Therefore, *planedefine* will only model the effects of biofouling and ignore potential effects from coating roughness and cracks for simplicity. Additionally, since the errors of modern pneumatic sensors are significantly less than the effects of marine biofouling, the sensor errors are also assumed to be negligible.

Using k and k_s values from [13], three types of noise distributions will be implemented into *planedefine* to simulate biofouling surface roughness. The first is a positive half of a Gaussian distribution with $\mu = 0 \text{ cm}$, $\sigma = 0.75 \text{ cm}$. Gaussian distributions are ubiquitous in nature and are commonly used to simulate real-world noise conditions. Next, a Uniform distribution will be

used with $\min = 0 \text{ cm}$, $\max = 1.5 \text{ cm}$. This will help test how well *IDS* handles evenly distributed and consistent noise levels. The third distribution will be a Poisson distribution with $\lambda = 0.75$. Several studies on marine fouling on surfaces treated with protective coating indicate that the accumulation of microorganisms follow a Poisson distribution [14]. A Poisson noise distribution can also better model the random attachment events of microorganisms to submerged surfaces. The three noise distributions employed are calibrated to produce comparable noise ranges, allowing for a better comparison of how the shapes of the distributions impact the results of *IDS*.

As seen in **Figure 7**, the noise is introduced along the sensor's vector $\bar{v}_{\text{sensor}}$. **Figure 9** provides an example of a Poisson noise distribution extending from the plane's surface given that $\bar{v}_{\text{sensor}}$ is normal to the real plane.

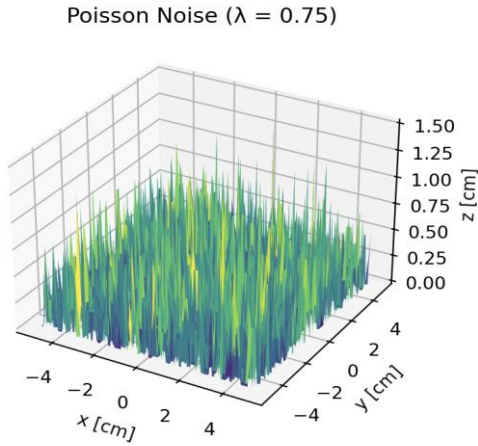


Figure 9: Simulating marine biofouling on the surface of a plane with a Poisson noise distribution.

5. PLANE IDENTIFICATION

The goal of *IDS* is to use the contact points generated by *planeldefine* and create a best-fit plane. This is achieved by minimising the sum of perpendicular squared distances between the plane and the points. This section will describe the general procedure of *IDS*'s script.

A plane can be completely determined if a point on the plane and its normal vector is known. The best-fit plane to a set of points will always pass through the centroid of the points. To prove this, let e be the sum of the residuals squared:

$$e = \sum_{i=1}^n ((\bar{p}_i - \bar{v}) \cdot \hat{n})^2 \quad (6)$$

Where vector $\bar{p}_i$ represents each given point in space, $\bar{v}$ represents an arbitrary point on the best-fit plane, $\hat{n}$ is the plane's unit normal, and n is the number of points. Expanding the summation,

$$e = \sum_{i=1}^n \hat{n}(\bar{p}_i \cdot \bar{p}_i) \hat{n}^T - 2 \sum_{i=1}^n \hat{n}(\bar{p}_i \cdot \bar{v}) \hat{n}^T + \sum_{i=1}^n \hat{n}(\bar{v} \cdot \bar{v}) \hat{n}^T \quad (7)$$

$\hat{n} \hat{n}^T = 1$ by definition. Now setting $\frac{de}{d\bar{v}} = 0$,

$$0 = -2 \sum_{i=1}^n \bar{p}_i + 2\bar{v} \sum_{i=1}^n 1 \quad (8)$$

$$\bar{v} = \frac{\sum_{i=1}^n \bar{p}_i}{\sum_{i=1}^n 1} = \frac{1}{n} \sum_{i=1}^n \bar{p}_i = \bar{p}_{\text{avg}} \quad (9)$$

Thus, the point $\bar{v}$ on the best-fit plane that minimises e is the centroid of the set of given points [15]. Therefore, the centroid must lie on the best-fit plane.

Next, the unit normal of the best-fit plane is to be determined. Let **A** be a $3 \times n$ matrix containing the set of contact points given by *planeldefine*:

$$\mathbf{A} = [\bar{p}_1 \quad \bar{p}_2 \quad \cdots \quad \bar{p}_n] \quad (10)$$

A principal component analysis, or PCA, is now performed on **A**. PCA is a statistical technique that identifies directions of maximum variance in a data set. This is important because a best-fit plane will span in directions of maximum variance to best capture the distribution of the points. The first step in PCA is to mean-centre the set of points. A matrix **B** of mean-centred points can be computed by subtracting a $3 \times n$ matrix containing columns of the centroid vector from **A**.

$$\mathbf{B} = \mathbf{A} - [\bar{p}_{\text{avg}} \quad \cdots \quad \bar{p}_{\text{avg}}] \quad (11)$$

This has the effect of shifting the data to the origin, making the centroid of the new set of points at $(0, 0, 0)$. Centred data allows PCA to capture the variance of the data without being influenced by the mean and absolute positions of the data.

Next, a singular value decomposition (SVD) is performed on **B** to obtain:

$$\mathbf{B} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

$$= [\bar{u}_1 \quad \bar{u}_2 \quad \bar{u}_3] \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix} \begin{bmatrix} \bar{v}_1^T \\ \bar{v}_2^T \\ \vdots \\ \bar{v}_n^T \end{bmatrix} \quad (12)$$

Where $\mathbf{U}$ is a 3×3 orthonormal matrix containing left singular vectors, $\mathbf{\Sigma}$ is a $3 \times n$ diagonal matrix containing singular values, and $\mathbf{V}$ is an $n \times n$ matrix containing right singular vectors, and $\sigma_1 > \sigma_2 > \sigma_3$. By nature of SVD, the vectors $\bar{u}_1$, $\bar{u}_2$, and $\bar{u}_3$ form an orthonormal basis and are defined as the principal components of the matrix $\mathbf{B}$. The first principal component $\bar{u}_1$ points in the direction of maximum variance of the data set. It has a corresponding singular value σ_1 , which represents how well $\bar{u}_1$ captures the variance of the dataset. $\bar{u}_2$ points in a second direction of maximum variance whilst being orthogonal to $\bar{u}_1$, with a corresponding singular value σ_2 . $\bar{u}_3$ then points in a third orthogonal direction of maximum variance. In summary, the principal components capture decreasing amounts of variance orthogonal to the previous principal component.

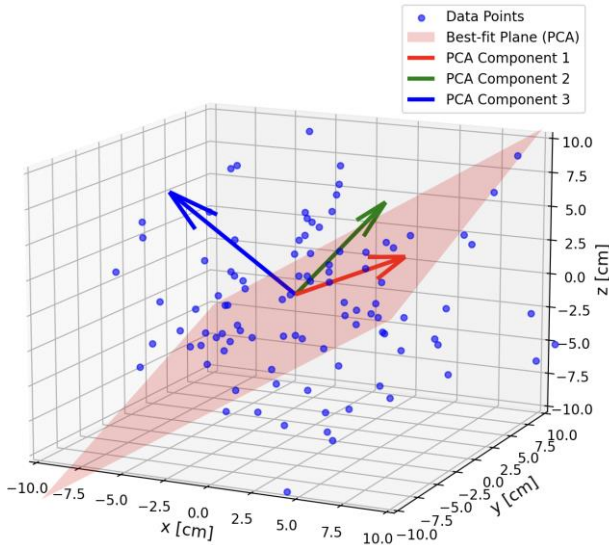


Figure 10: 3D plot of best-fit plane from a set of sample points using principal component analysis.

From **Figure 10** above, the first two principal components, PC1 and PC2, form a basis for the best-fit plane. They collectively explain as much variance as possible with two dimensions, providing the best 2D linear approximation to the data. The normal vector to this plane is therefore the third principal component $\bar{u}_3$ because it must be orthogonal to both $\bar{u}_1$ and $\bar{u}_2$.

Now, with both the normal vector and a point on the best-fit plane known, the plane is fully determined. Note that the plane obtained using PCA

is also a plane that minimises the sum of squared residuals [16]. By setting the plane to span the first two principal components, PCA effectively minimises the orthogonal distances between the plane and points, providing a solution that solves the least squares problem without explicitly optimising for it.

PCA was chosen for plane fitting over the traditional least squares method due to several reasons. Firstly, PCA offers a more computationally efficient approach because it does not require any matrix inversion or minimisation routine, which is advantageous when dealing with large data sets, or when the data is close to perfect. Additionally, PCA provides a straightforward way to identify the principal axes of variation, simplifying the process of fitting the plane. Moreover, it does not inherently assume any relationships between variables, making it a versatile tool for tasks such as plane fitting. By transforming the spatial coordinates into orthogonal principal components, PCA also eliminates any multicollinearity issues while improving stability and interpretability. **Figure 11** below depicts a best-fit plane provided by *IDS* using PCA.

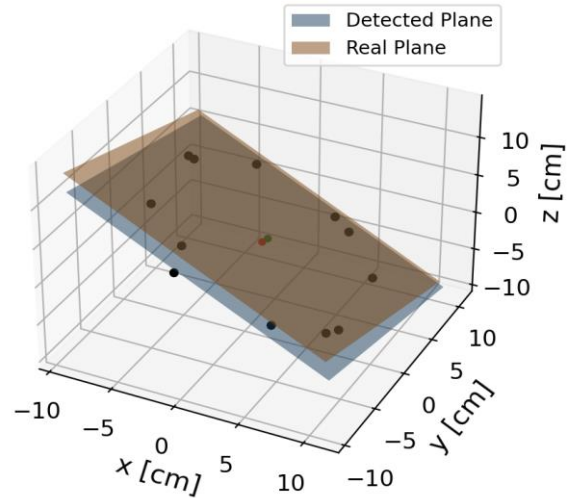


Figure 11: 3D plot of the real and fitted plane for $\alpha = 4$ and $\beta = 3$ with Poisson noise. The distance between the red and green points is δ (to be discussed in the next section).

5.1 Performance Metrics

After the best-fit plane is found, there are three performance metrics measured to quantify how well the fit is. The first metric is the angle difference θ between the normal vectors of the real plane and fitted plane. The second metric is determined by first starting at the centroid of the given contact points. A line through this centroid is created that is normal to the best-fit plane. This line will intersect

both the real and best-fit plane at some point. The distance between these two intersection points, δ , is the second metric. In the perfect case where the fitted plane and real plane coincide, $\theta = \delta = 0$.

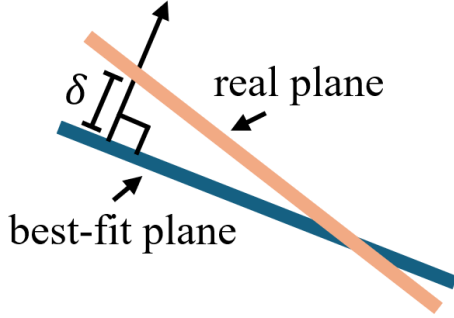


Figure 12: Diagram illustrating the performance metric δ .

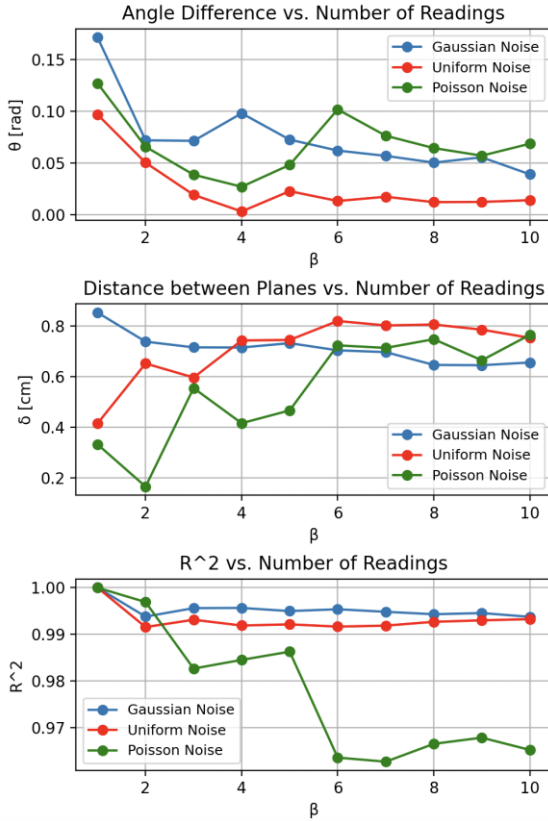
The third metric is the coefficient of determination, R^2 , for the fitted plane.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \quad (13)$$

where SS_{res} is the sum of squared residuals, and SS_{tot} is the total sum of squares. The coefficient of determination does not indicate how well the fitted plane matches the real plane, but rather represents how well it fits the measured set of points.

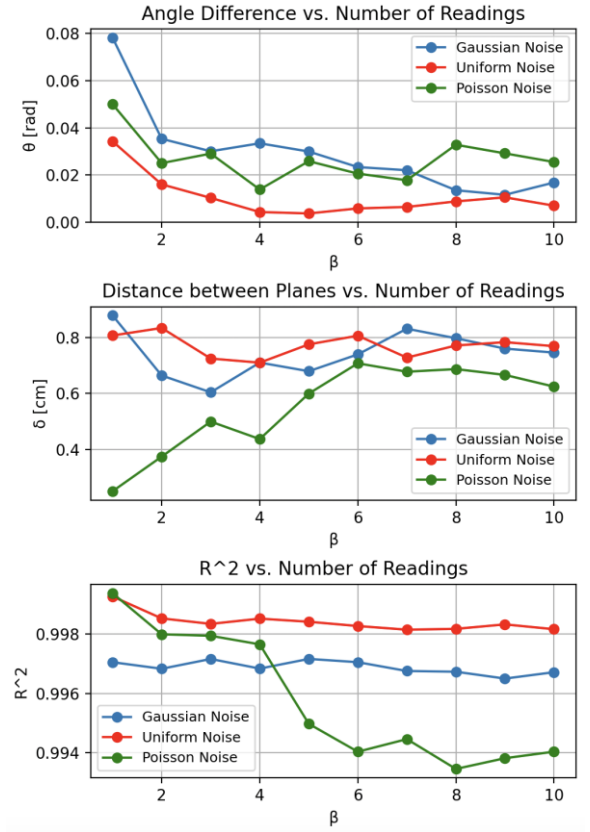
6. IDS RESULTS AND DISCUSSION

$$\alpha = 3, \beta \in \mathbb{Z}_{\{1,10\}}$$



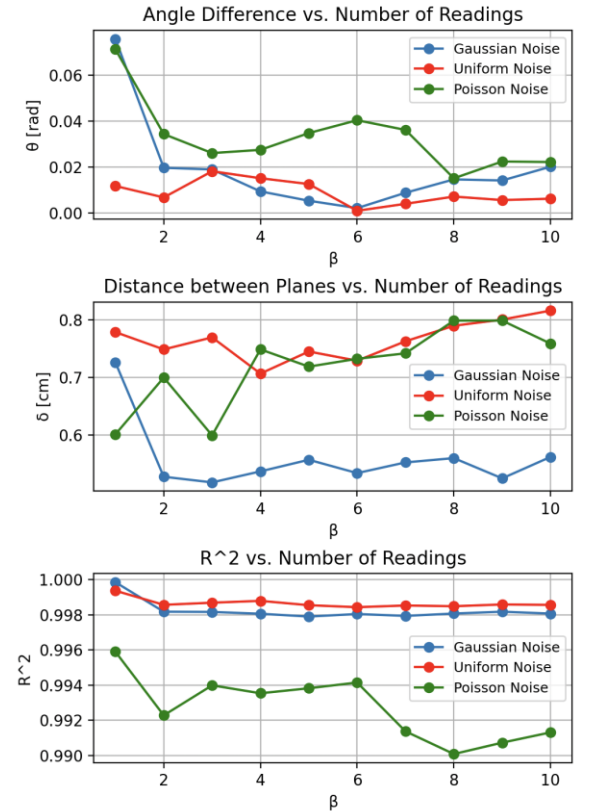
(a)

$$\alpha = 4, \beta \in \mathbb{Z}_{\{1,10\}}$$



(b)

$$\alpha = 5, \beta \in \mathbb{Z}_{\{1,10\}}$$



(c)

Figure 13 (a) – (c): Plots of the performance metrics θ , δ , and R^2 vs the number of sensor readings for (a) $\alpha = 3$, (b) $\alpha = 4$, and (c) $\alpha = 5$.

Figure 13 displays the performance metrics of the fitted plane based on a random real plane created by *planedefine*. From **Figure 13** (a) – (c), θ generally decreases as β increases. This occurs because as more contact points are measured, the best-fit plane is continuously improved and the normal vectors of the real and fitted plane align. Additionally, θ approaches zero at a faster rate for higher α because each measurement will contain more contact points, resulting in better fitted planes. In general, plots (b) can be considered as a ‘faster’ version of plots (a) because each measurement from the tactile sensor gathers one more piece of information when $\alpha = 4$ compared to when $\alpha = 3$. The same conclusion can be made for (c). Therefore, for higher α , the performance metrics will approach the steady-state values faster. For example, looking at (a) for Uniform noise, the angle difference between the planes was 0.05 rad after two readings. However, for $\alpha = 5$ with Uniform noise in (b), θ is already much less than 0.05 rad after two readings. It is true that this result could just be due to the random nature of the noise introduced, especially for low β . However, as shown in **Figure 13** (a) – (c), if sufficient measurements are obtained, larger α values require less readings to achieve the same quality of fit.

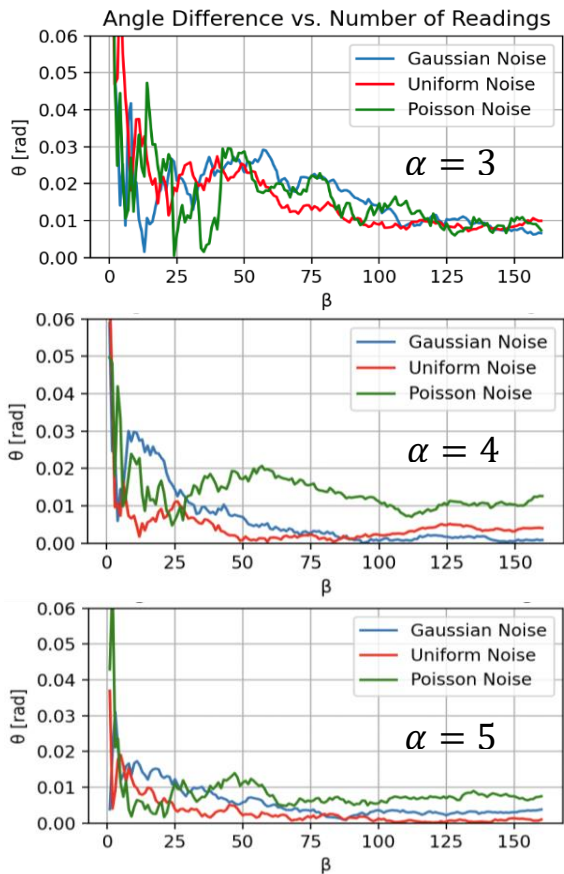


Figure 14: Plots of θ vs β for different values of α .

Figure 14 also shows that for a higher α , θ approaches zero much faster and stabilises much quicker.

Note that although θ approaches zero as β increases for all cases, δ does not approach zero. This is because when implementing the biofouling noise model, the average noise height on the plane is non-zero. For the previously discussed three noise distributions, the mean noise height for both the Uniform and Poisson distributions are 0.75 cm . For the Gaussian distribution, the mean of the half-normal distribution is given by, $\sigma \sqrt{\frac{2}{\pi}} = 0.75 \sqrt{\frac{2}{\pi}} = 0.598 \text{ cm}$. Thus, for very large β , the best-fit plane should become increasingly parallel to the real plane ($\theta \rightarrow 0$), while the distance between the planes δ should approach the mean height of each respective noise distribution. As seen in **Figure 15**, δ does indeed approach the respective means of the noise distributions. It also makes intuitive sense that δ never approaches zero because the measurements are taken on a surface with the noise distributions extending from the top of the plane.

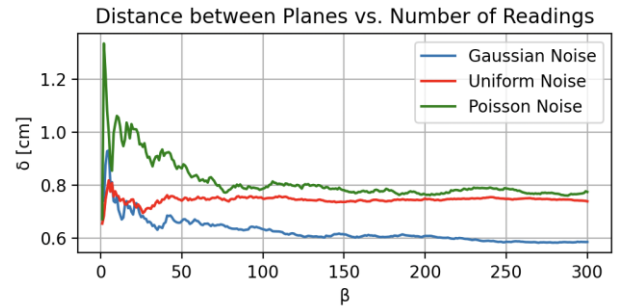


Figure 15: Plot of δ vs β for $\alpha = 3$. The steady-state value becomes more distinct for larger β .

For each measurement, the closer δ is to the mean of the noise distribution and the closer θ is to zero, the better the result of the measured plane will be because these are the steady-state values of *IDS*.

In terms of noise, plots (a) – (c) from **Figure 13** show that θ values subject to Uniform noise are generally lower compared to Gaussian noise for low values of β . This is due to the skewed nature of the gaussian distribution increasing the likelihood of a more tilted best-fit plane. However, δ values were generally lower in the presence of Gaussian noise compared to Uniform noise for low β . Additionally, the performance metrics when subject to the Poisson noise appear to fluctuate more than the other distributions. **Figures 14-15** also indicate that

the green Poisson curves take relatively more measurements to stabilize.

Out of all test cases in **Figure 13**, the highest angle difference recorded after 1 reading was $\theta = 0.18$ rad with Gaussian noise at $\alpha = 3$. The largest distance between planes recorded after 1 reading was $\delta = 0.89$ cm at $\alpha = 4$, again subject to Gaussian noise. However, the first reading values will vary significantly from trial to trial due to a low number of contact points.

Although θ will tend to zero as the number of readings are increased, this is very impractical. An accurate output is desired while also minimising the number of readings needed. A sufficiently appropriate angle difference value is defined as $\theta_a = 0.04 \text{ rad} \approx 2.3^\circ$. From **Figure 13** again, for $\alpha = 3$, only θ values for the Uniform noise stay under θ_a after three or more readings. The behaviour of θ when subject to Gaussian and Poisson noise fluctuate and end up greater than θ_a even after 10 readings. For the plots with $\alpha = 4$ and $\alpha = 5$, θ stays below θ_a after just two readings.

For δ , an appropriate value depends largely on the noise distribution introduced because δ will converge to the distribution's mean. The means were 0.598 cm , 0.75 cm and 0.75 cm for the Gaussian, Uniform, and Poisson distributions, respectively. Using an error of 5%, the appropriate δ value should be less than $\delta_a = 0.627 \text{ cm}$ when subject to Gaussian noise and less than $\delta_a = 0.788 \text{ cm}$ when subject to Uniform and Poisson noise. From **Figure 13**, when subject to Gaussian noise, $\delta < \delta_a$ after two or more readings only for the case when $\alpha = 5$. The same trend can be seen with Uniform and Poisson noise, with δ staying less than δ_a more frequently for higher α . Thus, $\alpha = 5$ will on-average achieve $\theta < \theta_a$ and $\delta < \delta_a$ quicker for the same number of readings β .

7. CONCLUSION

In this study, two programs were developed via Python: *planedefine* and *IDS*. These two programs are independent of each other but work together to simulate a tactile sensor environment, perform data collection, and data analysis to identify the relevant shape. In general, the results obtained from *IDS* across varying number of linear sensors, number of readings, and different noise distributions demonstrated accurate identification of planes in 3D

space. Analysis of the performance metrics also revealed that higher number of sensors per reading led to faster convergence in the identification script, yielding more accurate results. Different types of noise distributions also significantly affected performance, with *IDS* exhibiting lower θ values with Uniform noise and lower δ values with Gaussian noise. Poisson noise induced less stable behaviour across all metrics for varying β .

7.1 Limitations

Despite the promising results, there are certain limitations. Firstly, the simulations were conducted in idealized conditions with several assumptions made about the environment and sensor. Real-world applications will likely introduce additional complexities not accounted for in the models. For instance, variations in sensor calibration, environmental factors, and surface properties could impact the accuracy of shape identification. There could be errors in the positioning and motion of the tactile device, or non-negligible sensor errors not accounted for in the noise model. Additionally, *planedefine* may not fully capture the diversity and variability of noise encountered in real-world scenarios. Future research could address these limitations by incorporating more realistic noise models and experimental validations in real-world settings.

7.2 Further Work

There are several opportunities for further work to be done. Firstly, extending the scope of shape identification to address more complex geometries beyond planes can enhance the applicability of the program to more useful scenarios. The next two shapes to investigate can be spheres and cylinders. Moreover, investigating the integration of machine learning techniques, such as neural networks, could improve the robustness and adaptability of shape identification algorithms. Additionally, conducting experimental validations using physical tactile sensors in real-world environments would provide valuable insights into the performance and limitations of the proposed simulated methods. Overall, further research in this field holds the potential to advance the capabilities of tactile sensing systems and its underwater applications.

8. REFERENCES

- [1] S. Liu, D. Zhang, X. Fu, "Tactile Sensing for Soft Robotic Manipulators in 50 MPa

- Hydrostatic Pressure Environments”, *Advanced Intelligent Systems*, vol. 5, issue 12, Sep. 2023.
<https://doi.org/10.1002/aisy.202300296>
- [2] U. Martinez-Hernandez, “Tactile Sensors”, *Scholarpedia*, Jun. 2015.
<https://doi.org/10.21467/ajgr>
- [3] M. Lin, M. Vatani, J-W. Choi, S. Dilibal, E.D. Engeberg, “Compliant underwater manipulator with integrated tactile sensor for nonlinear force feedback control of an SMA actuation system”, *Sensors and Actuators A: Physical*, vol. 315, Nov. 2020.
<https://doi.org/10.1016/j.sna.2020.112221>
- [4] S. Caselli, C. Magnanini, F. Zanichelli, “Haptic object recognition with a dextrous hand based on volumetric shape representations”, *Multisensor Fusion and Integration for Intelligent Systems*, Oct. 1994. 10.1109/MFI.1994.398442
- [5] Schneider, J. Sturm, C. Stachniss, M. Reiser, H. Burkhardt, W. Burgard, “Object identification with tactile sensors using bag-of-features”, *Intelligent Robots and Systems*, Oct. 2009. 10.1109/IROS.2009.5354648
- [6] G. Heidemann, M. Schöpfer, “Dynamic tactile sensing for object identification”, *International Conference on Robotics and Automation*, vol. 1, pp. 813-18, Jan. 2004. 10.1109/ROBOT.2004.1307249
- [7] A. Greig, “Tactile Shape Identification”, *University College London*, 1992.
- [8] J.F. Røsjordet, G. Hovland, Methods for Experimentally Determining Stiffness of a Multi-Axis Machining Centre”, *Modeling, Identification and Control*, vol. 40, no. 1, pp. 11–25, Jan. 2019. 10.4173/mic.2019.1.2
- [9] Festo, “ISO cylinder to ISO 15552 with fixed cushioning and contactless displacement encoder”, *Festo*,
https://www.festo.com/gb/en/p/standards-based-cylinder-with-displacement-encoder-id_DDPC/?q=%7E%3AfestoSortOrderScored
- [10] S. Chen, H. Xu, F. Haseeb, W. Fan, Q. Wei, “A pneumatic–hydraulic hybrid actuator for underwater soft robot swimming and crawling”, *Sensors and Actuators A: Physical*, vol. 356, Jun. 2023.
<https://doi.org/10.1016/j.sna.2023.114284>
- [11] A. Pistone, C. Scolaro, A. Visco, “Mechanical Properties of Protective Coatings against Marine Fouling”, *Polymers*, Jan. 2021. 10.3390/polym13020173
- [12] T. Chivers, “The influence of surface roughness on fluid flow through cracks”, *Fatigue Fracture of Engineering Materials Structures*, vol. 25, issue 11, pp. 1095-1102, Oct. 2002. 10.1046/j.1460-2695.2002.00595.x
- [13] P.X. Zou, N. Ruiter, W.S.J. Uijttewaall, X.X. Chen, D.J. Peters, J.D. Bricker, “Experimental study of surface roughness effects on hydrodynamic characteristics of a submerged floating tunnel”, *Applied Ocean Research*, vol. 135, Jun. 2023.
<https://doi.org/10.1016/j.apor.2023.103557>
- [14] P. Jadidi, M. Zeinoddini, M. Soltanpour, A.P. Zandi, M.S. Seif, “Towards an understanding of marine fouling effects on VIV of circular cylinders: Aggregation effects”, *Ocean Engineering*, vol. 147, pp. 227-242, Jan. 2018.
<https://doi.org/10.1016/j.oceaneng.2017.10.037>
- [15] C.M. Brown, “Principal Axes and Best-Fit Planes, with Applications”, *University of Rochester*, Apr. 1976. <http://hdl.handle.net/1802/13747>
- [16] MATLAB Documentation, “Fitting an Orthogonal Regression Using Principal Components Analysis”, *Mathworks*, 2024,
<https://uk.mathworks.com/help/stats/fitting-an-orthogonal-regression-using-principal-components-analysis.html>